

Titanic Classification Modelling with PySpark MLlib

Neha Vadapally



Implement machine learning algorithms for data analytics	2
1. Introduction of Problem	3
1.1 Background to Classification in Data Analytics	3
1.2 Problem Definition and Objectives	3
1.3 Dataset Overview (Titanic Survival Dataset)	3
1.4 Problem Decomposition and Analytical Workflow	4
2. Approaches	5
2.1 Data Pre-processing	5
2.1.1 Data Cleaning and Handling Missing Values	5
2.1.2 Data Normalisation	6
2.1.3 Data Visualization	6
2.1.4 Exploratory Data Analysis and Initial Insights	6
2.2 Feature Extraction and Feature Selection	7
2.2.1 Selection of Relevant Attributes	7
2.2.2 Encoding of Categorical Variables	7
2.2.3 Feature Vector Construction	8
2.3 Classification Models	9
2.3.1 Train - Test Split Strategy	9
2.3.2 Classification Algorithms	10
3. Results Analysis	10
3.1 Exploratory Data Analysis Results	11
3.2 Model Performance Metrics	11
3.3 Confusion Matrix Analysis	12
3.4 Comparative Model Evaluation	13
4. Discussion and conclusions	13
4.1 Key Findings and Insights	13
4.2 Model Evaluation and Limitations	13
4.3 Improvements and Future Work	14
References and Bibliography	14
Appendix	16

Implement machine learning algorithms for data analytics

1. Introduction of Problem

1.1 Background to Classification in Data Analytics

Classification is a basic concept in data analysis and it has been extensively used in areas where different categories of outcomes need to be forecasted based on past data (Bishop, 2006). In supervised classification, labelled examples are employed to make inferences about the decision boundary that can be generalized to new, unseen examples. These techniques are extremely helpful in real-world applications involving real data that is noisy, has missing values and has different types of variables (Hastie, Tibshirani and Friedman, 2009).

In applied analytics, classification is dependent on interpretability, robustness and error rates rather than accuracy. According to Bishop (2006), the assumption underlying the classification technique, representation and preprocessing of features all affect the accuracy of a classification model.

1.2 Problem Definition and Objectives

The objective of this study is to develop a passenger survivability forecast using supervised machine learning algorithms offered by the PySpark MLlib package.

- The target variable 'Survived' is a binary variable, which classifies the passengers into two categories: the survivors and the non-survivors.
- Other objectives of the study are:
 - Exploration of the quality of the data.
 - Detection of trends using exploratory data analysis techniques.
 - Comparing models with various theoretical assumptions.
 - Evaluation of the models using relevant statistical criteria.

This is according to Domingos' (2012) criteria for machine learning applications.

1.3 Dataset Overview (Titanic Survival Dataset)

```
df = spark.read.csv("/kaggle/input/c2-titanic/TitanicData1.csv",
                    header=True,
                    inferSchema=True)
```

```
df.show(5)
df.printSchema()
```

PassengerId	Pclass	Name	Sex	Age	SibSp	Parch	Ticket	Fare	Cabin	Embarked	Survived
1	3	Braund, Mr. Owen ...	male	22.0	1	0	A/5 21171	7.25	NULL	S	0
2	1	Cumings, Mrs. Joh...	female	38.0	1	0	PC 17599	71.2833	C85	C	1
3	3	Heikkinen, Miss. ...	female	26.0	0	0	STON/O2. 3101282	7.925	NULL	S	1
4	1	Futrelle, Mrs. Ja...	female	35.0	1	0	113803	53.1	C123	S	1
5	3	Allen, Mr. Willia...	male	35.0	0	0	373450	8.05	NULL	S	0

only showing top 5 rows

root

```
-- PassengerId: integer (nullable = true)
-- Pclass: integer (nullable = true)
-- Name: string (nullable = true)
-- Sex: string (nullable = true)
-- Age: double (nullable = true)
-- SibSp: integer (nullable = true)
-- Parch: integer (nullable = true)
-- Ticket: string (nullable = true)
-- Fare: double (nullable = true)
-- Cabin: string (nullable = true)
-- Embarked: string (nullable = true)
-- Survived: integer (nullable = true)
```

Figure 1: Raw Titanic data snapshot

From the above figure, the dataset includes various demographic, socioeconomic and travel-related information, such as age, gender, class, fare and embark, related to the Titanic disaster. This includes missing values and class imbalance, as depicted in Figure 16 and thus needs to be pre-processed, as it affects the results of the classification (Han, Kamber and Pei, 2012).

1.4 Problem Decomposition and Analytical Workflow

To solve the problem in a systematic way, the problem was divided into the following steps:

1. Data Cleaning and Preparation
2. Exploratory Data Analysis and Visualization
3. Feature Extraction and Transformation
4. Model Training and Evaluation
5. Comparative Analysis and Critical Reflection

The steps are informed by existing data analysis and machine learning pipelines and are not considered to be isolated computing processes, as argued by Tukey in 1977.

2. Approaches

2.1 Data Pre-processing

2.1.1 Data Cleaning and Handling Missing Values

Little and Rubin (2019) recommend that mean imputed values be used for the age feature and Embarked feature, using the most common values. All algorithms require complete information for the features and the missing values must be handled (Han, Kamber, & Pei, 2012).

```
df.select([
  (df[c].isNull().cast("int")).alias(c)
  for c in df.columns
]).groupBy().sum().show()
```

sum(PassengerId)	sum(Pclass)	sum(Name)	sum(Sex)	sum(Age)	sum(SibSp)	sum(Parch)	sum(Ticket)	sum(Fare)	sum(Cabin)	sum(Embarked)	sum(Survived)
0	0	0	0	177	0	0	0	0	687	2	0

Figure 2: Missing values count summary

Dealing with the missing values shown in Figure 2 and 3 is important, considering that most machine learning algorithms cannot function properly on a dataset that has missing values (Han, Kamber, & Pei, 2012).

```
from pyspark.sql.functions import col, when

mean_age = selected_df.select("Age").groupBy().mean().first()[0]

clean_df = selected_df \
    .withColumn(
        "Age",
        when(col("Age").isNull(), mean_age).otherwise(col("Age"))
    ) \
    .withColumn(
        "Embarked",
        when(col("Embarked").isNull(), "S").otherwise(col("Embarked"))
    )

clean_df.select([
  (clean_df[c].isNull().cast("int")).alias(c)
  for c in clean_df.columns
]).groupBy().sum().show()
```

sum(Survived)	sum(Pclass)	sum(Sex)	sum(Age)	sum(SibSp)	sum(Parch)	sum(Fare)	sum(Embarked)
0	0	0	0	0	0	0	0

Figure 3: Post imputation validation summary

2.1.2 Data Normalisation

To ensure stability and convergence of the model, the Logistic Regression model was trained on the normalized values of the features (Aggarwal, 2015), as shown in Figure 8 and 17. However, the Naive Bayes model was trained on non-normalized values, as shown in Figure 18, to compare the methodologies.

2.1.3 Data Visualization

In order to enable the exploratory analysis, the visualization method was used to enable the understanding of the different correlations with the survival rates. The plots in Figure 4, illustrate the survival rates based on gender and class enabled the intuitive detection of trends, which could not have otherwise been understood, thus making the use of visualization an extremely crucial step in the initial phases of the study (Few, 2012).

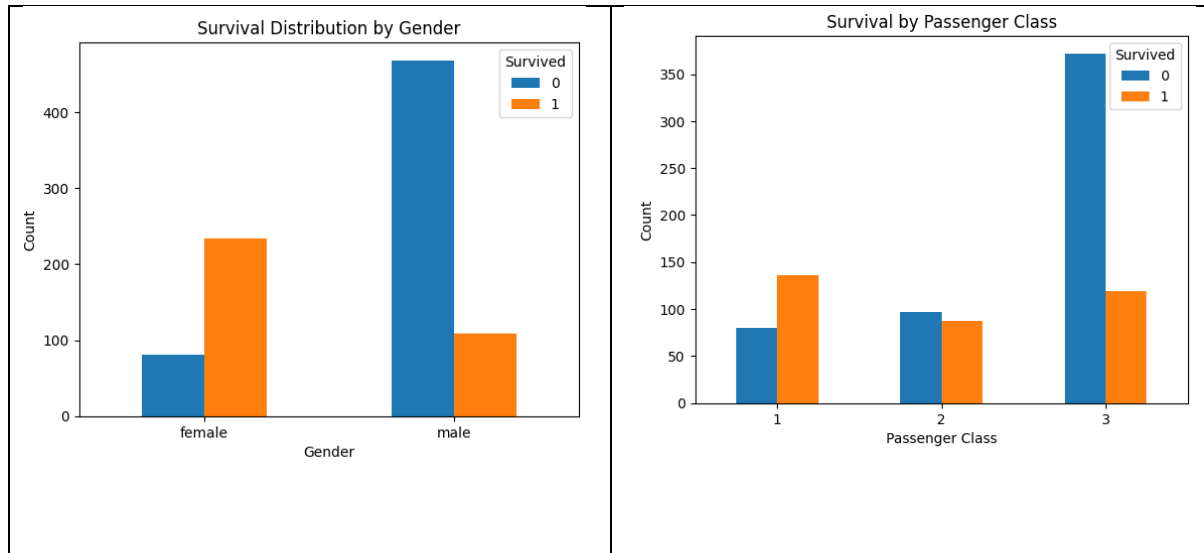


Figure 4: Imbalance in survival rates by gender and Passenger Class

2.1.4 Exploratory Data Analysis and Initial Insights

- Females and high-class passengers have a high survival rate, but third-class passengers don't fare too well.

- According to Sokolova and Lapalme (2009), there is a class imbalance that favors non-survivors and it needs to be interpreted with care.
- From the Box plot Figures 19 and 20, Fare is right-skewed due to socioeconomic factors hence retained, but age has a minimal impact on imputation.

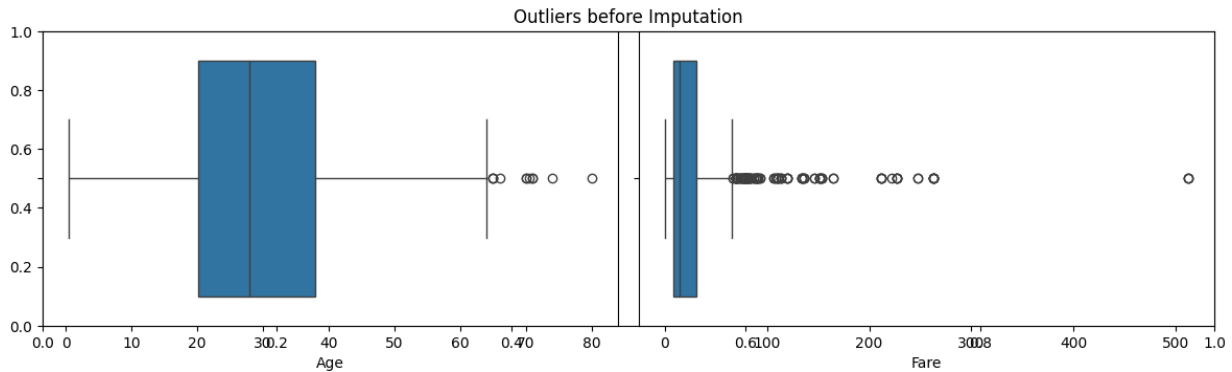


Figure 5: Unprocessed variables exhibit wide and uneven spread

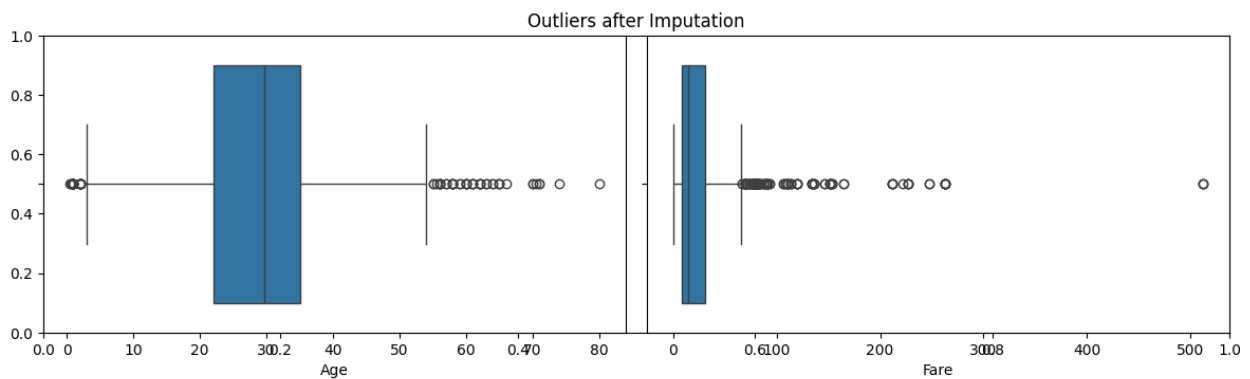


Figure 6: Imputation yields smoother distributions with stable spread

2.2 Feature Extraction and Feature Selection

2.2.1 Selection of Relevant Attributes

However, the relevance of the domain and the exploratory findings also have an impact on feature selection. The class, demographic and socioeconomic status features are kept and the unusual and unwanted features are deleted. Feature selection helps to reduce noise and generalise the learning process by focussing on important features (Guyon and Elisseeff 2003).

2.2.2 Encoding of Categorical Variables

The index-based encoding technique and one-hot encoding strategy are used for encoding the category variables (See Figure 7). When there is no need to impose any arbitrary order

on the classes and variables are nominal, the approach for encoding category variables for classification problems is appropriate (Kuhn and Johnson, 2013).

```
from pyspark.ml.feature import StringIndexer, OneHotEncoder

sex_indexer = StringIndexer(
    inputCol="Sex",
    outputCol="SexIndex",
    handleInvalid="keep"
)

embarked_indexer = StringIndexer(
    inputCol="Embarked",
    outputCol="EmbarkedIndex",
    handleInvalid="keep"
)

encoder = OneHotEncoder(
    inputCols=["SexIndex", "EmbarkedIndex"],
    outputCols=["SexVec", "EmbarkedVec"]
)
```

Figure 7: Encoding prepares variables for modelling

2.2.3 Feature Vector Construction

In Figure 8, the encoded category variables and the numerical variables are concatenated to ensure that training and testing data are represented. Using the pipeline for creating features improves the repeatability of the model and avoids data leakage during testing.

```

from pyspark.ml.feature import VectorAssembler

assembler = VectorAssembler(
    inputCols=[
        "Pclass",
        "Age",
        "SibSp",
        "Parch",
        "Fare",
        "SexVec",
        "EmbarkedVec"
    ],
    outputCol="features"
)

```

```

from pyspark.ml.feature import StandardScaler
scaler = StandardScaler(
    inputCol="features",
    outputCol="scaledFeatures",
    withMean=True,
    withStd=True
)

```

Figure 8: Combining attributes into analytic vectors

2.3 Classification Models

2.3.1 Train - Test Split Strategy

The data was split into a set of training and a set of testing instances in a ratio of 70:30 as shown in Figure 9. This technique provides an unbiased estimate of the generalization ability while retaining enough instances in the set for learning (Hastie, Tibshirani and Friedman, 2009).

```

train_df, test_df = clean_df.randomSplit([0.7, 0.3], seed=42)

print("Training records:", train_df.count())
print("Testing records:", test_df.count())

```

```

Training records: 659
Testing records: 232

```

Figure 9: Data partition for model evaluation

2.3.2 Classification Algorithms

- Logistic regression was employed as a discriminative classifier for scaled data. This technique is commonly utilized due to its probabilistic nature and ease of interpretation, especially when dealing with binary classification problems (Hosmer, Lemeshow and Sturdivant, 2013).
- Naive Bayes was employed as a probabilistic baseline classifier. This technique, even though it has strong conditional independence assumptions, has been known to perform better than optimization-based techniques in many instances, especially when dealing with structured data (Murphy, 2012).

The ROC-AUC metric will be utilized to compare Logistic Regression, while metrics such as confusion matrix, accuracy, precision, recall and F1-score will be employed to compare the performance of Naive Bayes.

3. Results Analysis

3.1 Exploratory Data Analysis Results

The exploratory data analysis indicated that there was a significant survival difference based on gender and passenger class, thus emphasizing their significance. The class imbalance problem, on the other hand, highlighted the need to conduct an in-depth analysis based on various performance parameters. (Figure 4,5 and 6)

3.2 Model Performance Metrics

According to the data, the performance of the Logistic Regression model was better than the Naive Bayes model because the former achieved better precision and recall as shown in Figure 11. According to Powers (2011), the use of various parameters is essential while evaluating the performance of the model, especially when class imbalance problems are identified. Precision alone may not be adequate. In Figure 10, the ROC AUC score was also 0.85, indicating that the model has high discriminative power for all classification levels.

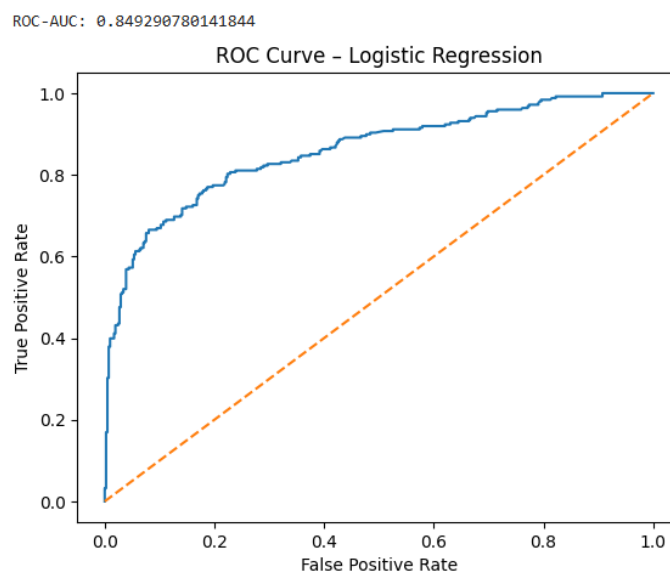


Figure 10: Curve indicates effective class separation

```
Logistic Regression Metrics
precisionByLabel: 0.8358
recallByLabel: 0.8116
f1: 0.7937

Naive Bayes Metrics
precisionByLabel: 0.6928
recallByLabel: 0.8333
f1: 0.6678
```

Figure 11: Comparative metrics favour logistic regression model

3.3 Confusion Matrix Analysis

From Figures 26 and 27, the analysis of the confusion matrix, detailed information about the error pattern was obtained. A higher rate of true positives was obtained by the Logistic Regression model and Naive Bayes had a higher tendency to incorrectly classify the survivors. This is in line with Naive Bayes' independence assumptions, which are unlikely to hold in this dataset (Murphy, 2012).

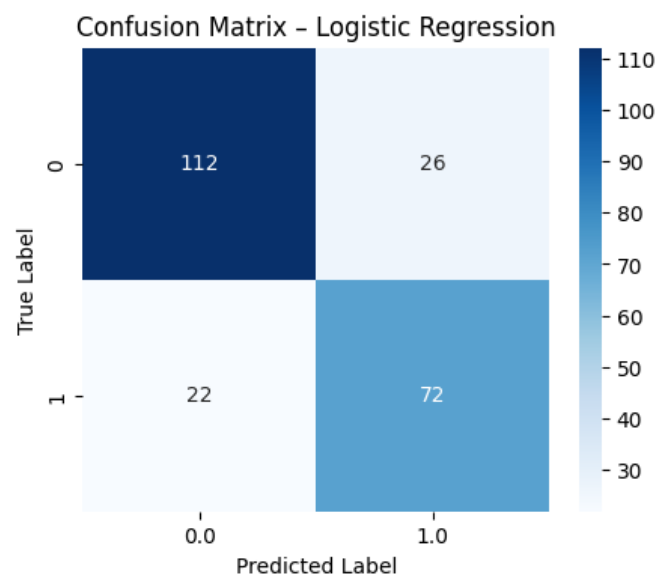


Figure 12: High true-positive and true-negative rates indicate reliable classification

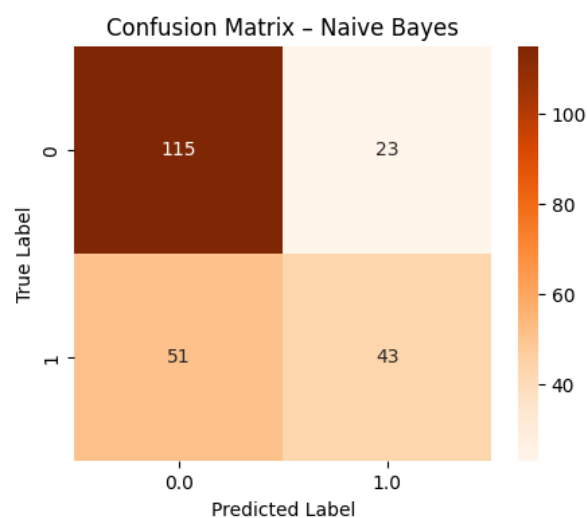


Figure 13: Frequent false negatives reveal weaker discriminative power

3.4 Comparative Model Evaluation

Comparison of model performance confirmed that Logistic Regression outperformed Naive Bayes in most instances. The mistake pattern analysis confirmed that mistakes were typical in passenger demographics, suggesting feature limitations rather than mistakes.

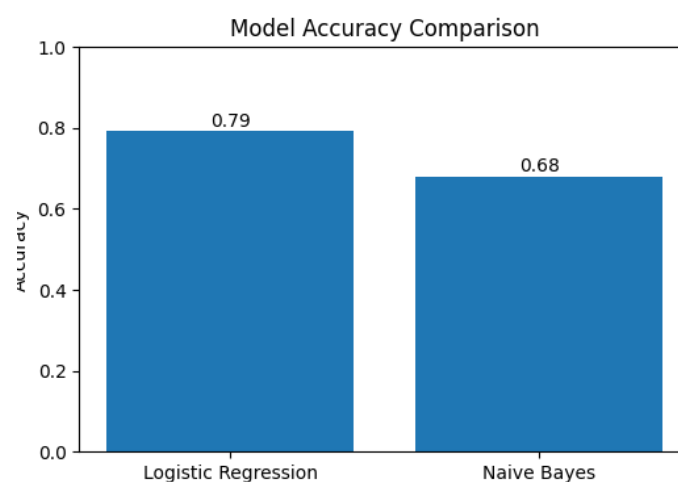


Figure 14: Accuracy difference (0.79 vs 0.68) demonstrates clear model advantage

4. Discussion and conclusions

4.1 Key Findings and Insights

The strongest factors affecting survival turned out to be gender and class. The logistic regression algorithm was found to perform better than other techniques because it can handle feature relationships without making strong independence assumptions.

4.2 Model Evaluation and Limitations

- Model limitations arise from class imbalance reducing minority-class recall, simple imputations distorting distributions and linear model assumptions restricting nonlinear pattern capture.

- In Figure 15, misclassifications are mostly false positives, indicating overlapping survivor and non-survivor feature characteristics that limit class separability.

```
lr_predictions \
  .filter(col("Survived") != col("prediction")) \
  .select("Survived", "prediction", "scaledFeatures") \
  .show(10)
```

Survived	prediction	scaledFeatures
0	1.0	[-1.6667172731712...]
0	1.0	[-1.6667172731712...]
0	1.0	[-1.6667172731712...]
0	1.0	[-1.6667172731712...]
0	1.0	[-1.6667172731712...]
0	1.0	[-1.6667172731712...]
0	1.0	[-1.6667172731712...]
0	1.0	[-1.6667172731712...]
0	1.0	[-1.6667172731712...]
0	1.0	[-1.6667172731712...]

only showing top 10 rows

Figure 15: Misclassified cases cluster around ambiguous feature patterns

4.3 Improvements and Future Work

Future work may also explore the use of other classifiers like decision trees or ensemble methods and better feature engineering and hyperparameter tuning. These have been found to help in improving the robustness and accuracy of difficult classification problems (Dieterich, 2000).

References and Bibliography

Bishop, C.M. (2006) Pattern Recognition and Machine Learning. New York: Springer. Available at: <https://link.springer.com/book/9780387310732> (Accessed: 29 January 2026).

Few, S. (2012) Show Me the Numbers: Designing Tables and Graphs to Enlighten. 2nd edn. Burlingame, CA: Analytics Press. Available at: <https://www.analyticspress.com/smtm.php> (Accessed: 29 January 2026).

Guyon, I. and Elisseeff, A. (2003) 'An introduction to variable and feature selection', *Journal of Machine Learning Research*, 3, pp. 1157–1182. Available at: <https://doi.org/10.1162/153244303322753616> (Accessed: 29 January 2026).

Hastie, T., Tibshirani, R. and Friedman, J. (2009) *The Elements of Statistical Learning: Data Mining, Inference and Prediction*. 2nd edn. New York: Springer. Available at: <https://link.springer.com/book/10.1007/978-0-387-84858-7> (Accessed: 29 January 2026).

Hosmer, D.W., Lemeshow, S. and Sturdivant, R.X. (2013) *Applied Logistic Regression*. 3rd edn. Hoboken, NJ: Wiley. Available at: <https://doi.org/10.1002/9781118548387> (Accessed: 29 January 2026).

Kuhn, M. and Johnson, K. (2013) *Applied Predictive Modeling*. New York: Springer. Available at: <https://doi.org/10.1007/978-1-4614-6849-3> (Accessed: 29 January 2026).

Little, R.J.A. and Rubin, D.B. (2019) *Statistical Analysis with Missing Data*. 3rd edn. Hoboken, NJ: Wiley. Available at: <https://www.wiley.com/en-us/Statistical+Analysis+with+Missing+Data%2C+3rd+Edition-p-00026061> (Accessed: 29 January 2026).

Murphy, K.P. (2012) *Machine Learning: A Probabilistic Perspective*. Cambridge, MA: MIT Press. Available at: <https://mitpress.mit.edu/9780262018029/machine-learning/> (Accessed: 29 January 2026).

Powers, D.M.W. (2011) 'Evaluation: From precision, recall and F-measure to ROC, informedness, markedness and correlation', *Journal of Machine Learning Technologies*, 2(1), pp. 37–63. Available at: <https://arxiv.org/abs/2010.16061> (Accessed: 29 January 2026).

Sokolova, M. and Lapalme, G. (2009) 'A systematic analysis of performance measures for classification tasks', *Information Processing and Management*, 45(4), pp. 427–437. Available at: <https://doi.org/10.1016/j.ipm.2009.03.002> (Accessed: 29 January 2026).

Tukey, J.W. (1977) *Exploratory Data Analysis*. Reading, MA: Addison-Wesley. Available at: https://openlibrary.org/books/OL4877620M/Exploratory_data_analysis (Accessed: 29 January 2026).

Aggarwal, C.C. (2015) *Data Mining: The Textbook*. Cham: Springer. Available at: <https://doi.org/10.1007/978-3-319-14142-8> (Accessed: 29 January 2026).

Appendix

```
df.describe().show()
```

26/01/29 23:11:09 WARN SparkStringUtils: Truncated the string representation of a plan since it was too large. This behavior can be adjusted by setting 'spark.sql.debug.maxToStringFields'.

summary	PassengerId	Pclass	Name	Sex	Age	SibSp	Parch	Ticket	Fare	Cabin	Embarked	Survived
count	891	891	891	891	714	891	891	891	891	204	889	891
mean	446.0	2.308641975388642	NULL	NULL	29.69911764705882	0.5238078563411896	0.38159371492704824	260318.54916792738	32.2042079685746	NULL	NULL	0.3838383838383838
stddev	257.3538420152301	0.8360712409770491	NULL	NULL	14.52649732334035	1.1027434322934315	0.0860572211299488	471609.26868834975	49.69342859718009	NULL	NULL	0.48659245426485753
min	1	1	Andersson, Mr. A...	female	0.42	0	0	110152	0.0	A10	C	0
max	891	3	van Heckebeke, Mr...	male	80.0	0	6	WE/P 5735	512.3292	T	S	1

Figure 16: Descriptive statistics reveal uneven feature spread

```
from pyspark.ml.classification import LogisticRegression
from pyspark.ml import Pipeline

lr = LogisticRegression(
    labelCol="Survived",
    featuresCol="scaledFeatures"
)

lr_pipeline = Pipeline(stages=[
    sex_indexer,
    embarked_indexer,
    encoder,
    assembler,
    scaler,
    lr
])

lr_model = lr_pipeline.fit(train_df)
lr_predictions = lr_model.transform(test_df)
```

Figure 17: Logistic Regression with scaled features

```
from pyspark.ml.classification import NaiveBayes

nb = NaiveBayes(
    labelCol="Survived",
    featuresCol="features"
)

nb_pipeline = Pipeline(stages=[
    sex_indexer,
    embarked_indexer,
    encoder,
    assembler,
    nb
])

nb_model = nb_pipeline.fit(train_df)
nb_predictions = nb_model.transform(test_df)
```

Figure 18: Naive Bayes with pipeline features